

Grupo 6

Servicio de Despacho y Logística

Briefing Técnico para el Equipo

Documento de alineación previo al desarrollo

Proyecto: Marketplace en la Nube
Asignatura: Arquitectura de Software
Fecha: 11 de Junio de 2026
Versión: 1.0 — Revisada y corregida

! Antes de leer

Este documento corrige inconsistencias detectadas en el briefing original y agrega elementos faltantes. **Léanlo completo antes de escribir una sola línea de código.**

Índice

1. Contexto del Negocio	2
2. Grupos con los que Interactuamos	2
3. Ciclo de Vida del Envío (Estados)	3
4. Contrato REST (API de Shipments)	3
4.1. Base URL	3
4.2. POST /shipments — Crear un despacho	3
4.3. GET /shipments/{shipment_id} — Consultar estado	4
4.4. PATCH /shipments/{shipment_id} — Actualizar estado	4
5. Contrato de Eventos	5
5.1. Formato estándar del evento	5
5.2. Catálogo de eventos	6
6. Conceptos Técnicos Obligatorios	6
7. Hoja de Ruta por Fases	7
8. Stack Tecnológico Propuesto	7
9. Esquema de Base de Datos	8
10. Diagrama de Arquitectura	8
11. Resumen de Correcciones Aplicadas	8
12. Próximos Pasos Inmediatos	9

1. Contexto del Negocio

Nuestro servicio es el equivalente interno a una empresa de courier (Starken, Chilexpress, Blue Express) dentro del ecosistema del Marketplace. **No vendemos productos ni procesamos pagos.** Nuestra responsabilidad se limita a tres acciones:

1. **Crear un despacho:** Cuando un cliente compra exitosamente, el sistema de pedidos nos avisa para que generemos una orden de envío (*Shipment*).
2. **Hacer seguimiento:** Mantener en nuestra base de datos el estado actual de cada paquete.
3. **Notificar cambios:** Avisar al resto del ecosistema cuando ocurran transiciones de estado.

2. Grupos con los que Interactuamos

⚠ Corrección: Falta G1 en el briefing original

El briefing original solo mencionaba G5, G9 y G10 como aliados, pero en la descripción de endpoints se indica que **el Frontend (G1)** también consumirá nuestro **GET** para mostrar el tracking al usuario. Debemos incluirlos.

Grupo	Nombre	Relación con nosotros	Dirección
G1	Frontend	Consume GET <code>/shipments/{id}</code> para mostrar tracking al usuario final.	Ellos → Nosotros
G5	Pedidos	Llama a POST <code>/shipments</code> cuando un pedido se confirma y paga. Es nuestro principal cliente .	Ellos → Nosotros
G9	Notificaciones	Escucha nuestros eventos (ShipmentCreated , ShipmentInTransit , ShipmentDelivered , etc.) para enviar correos/notificaciones al usuario.	Nosotros → Ellos
G10	Reportería	Consume nuestros datos (vía eventos o API) para dashboards de rendimiento: entregas a tiempo, tasa de fallos, etc.	Nosotros → Ellos

Cuadro 1: Matriz de interacciones del Grupo 6.

💡 Consejo

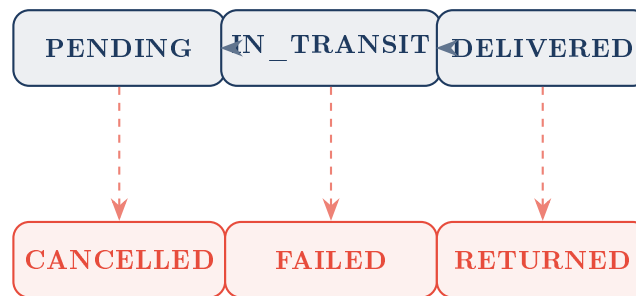
Acción inmediata: Contactar a los coordinadores de G5 y G9 para acordar los contratos JSON. Propuesta de mensaje:

“Hola, somos del Grupo 6 (Despachos). Necesitamos definir juntos: (1) qué datos nos envían al crear un pedido, y (2) qué datos necesitan recibir en las notificaciones de cambio de estado.”

3. Ciclo de Vida del Envío (Estados)

⚠ Corrección: Estados faltantes

El briefing original solo contemplaba tres estados lineales: `PENDING` → `IN_TRANSIT` → `DELIVERED`. Esto es insuficiente para cubrir escenarios de error y la rúbrica de **Manejo de Errores**. Se agregan los estados `CANCELLED`, `FAILED` y `RETURNED`.



Estado	Tipo	Descripción
PENDING	Normal	Despacho creado, esperando recolección.
IN_TRANSIT	Normal	Paquete en camino hacia la dirección de destino.
DELIVERED	Final	Paquete entregado exitosamente al cliente.
CANCELLED	Error	Despacho cancelado antes de salir a reparto (ej: pedido anulado por G5).
FAILED	Error	Intento de entrega fallido (dirección incorrecta, destinatario ausente).
RETURNED	Error	Paquete devuelto al origen tras entrega fallida o solicitud del cliente.

Cuadro 2: Estados del ciclo de vida de un envío.

4. Contrato REST (API de Shipments)

⚠ Corrección: Endpoint PATCH faltante

El briefing original solo definía `POST` y `GET`. Sin embargo, para actualizar el estado del envío (por ejemplo, cuando la simulación de la Fase 4 avanza el paquete de `IN_TRANSIT` a `DELIVERED`), se necesita un endpoint `PATCH`. Se agrega a continuación.

4.1 Base URL

```
https://<nombre-del-servicio>.onrender.com/api/v1
```

4.2 POST /shipments — Crear un despacho

Quién lo llama: Grupo 5 (Pedidos).

Request Body

```
{
  "order_id": "ORD-20260611-001",
  "customer_name": "María González",
  "address": "Av. Providencia 1234, Depto 56",
  "city": "Santiago",
  "weight_kg": 3.2
}
```

Response — 201 Created

```
{
  "shipment_id": "SHP-a1b2c3d4",
  "order_id": "ORD-20260611-001",
  "status": "PENDING",
  "created_at": "2026-06-11T14:00:00Z",
  "estimated_delivery": "2026-06-14T18:00:00Z"
}
```

Errores posibles

Código	Nombre	Cuándo
400	Bad Request	Faltan campos obligatorios o el JSON es inválido.
409	Conflict	Ya existe un shipment para ese <code>order_id</code> .
422	Unprocessable Entity	Datos presentes pero inválidos (ej: <code>weight_kg</code> negativo).

4.3 GET /shipments/{shipment_id} — Consultar estado

Quién lo llama: Grupo 1 (Frontend) y Grupo 5 (Pedidos).

Response — 200 OK

```
{
  "shipment_id": "SHP-a1b2c3d4",
  "order_id": "ORD-20260611-001",
  "status": "IN_TRANSIT",
  "customer_name": "Maria Gonzalez",
  "address": "Av. Providencia 1234, Depto 56",
  "city": "Santiago",
  "weight_kg": 3.2,
  "created_at": "2026-06-11T14:00:00Z",
  "updated_at": "2026-06-11T14:30:00Z",
  "estimated_delivery": "2026-06-14T18:00:00Z"
}
```

Errores posibles

4.4 PATCH /shipments/{shipment_id} — Actualizar estado

Quién lo llama: Nuestro propio servicio (simulación automática de la Fase 4) o un endpoint administrativo.

Código	Nombre	Cuándo
404	Not Found	No existe un shipment con ese ID.

Request Body

```
{
  "status": "DELIVERED"
}
```

Response — 200 OK

```
{
  "shipment_id": "SHP-a1b2c3d4",
  "status": "DELIVERED",
  "updated_at": "2026-06-11T15:00:00Z"
}
```

Errores posibles

Código	Nombre	Cuándo
400	Bad Request	Estado enviado no es válido.
404	Not Found	No existe un shipment con ese ID.
409	Conflict	Transición de estado inválida (ej: de DELIVERED a PENDING).

Nota

Las transiciones válidas son las mostradas en el diagrama de la Sección 3. Cualquier otra transición debe rechazarse con un **409 Conflict**.

5. Contrato de Eventos

Corrección: Mecanismo de eventos no especificado

El briefing original habla de “publicar eventos” y “emitir mensajes” sin aclarar la tecnología. Para un proyecto desplegado en Render (tier gratuito), las opciones realistas son:

- **Webhooks HTTP (recomendado):** Nuestro servicio hace un **POST** a una URL registrada del G9 cada vez que cambia un estado.
- **Polling:** G9/G10 consultan periódicamente nuestro **GET**. Menos eficiente pero más simple.

Decisión del equipo: Debemos acordar esto con G9 antes del 16 de Junio.

5.1 Formato estándar del evento

Cada evento que publiquemos debe seguir esta estructura:

```
{
  "event_type": "ShipmentDelivered",
  "event_id": "evt-uuid-autogenerado",
  "timestamp": "2026-06-11T15:00:00Z",
  "payload": {
    "shipment_id": "SHP-a1b2c3d4",
    "order_id": "ORD-20260611-001",
    "previous_status": "IN_TRANSIT",
    "new_status": "DELIVERED",
    "customer_name": "Maria Gonzalez",
    "city": "Santiago"
  }
}
```

5.2 Catálogo de eventos

event_type	Transición	Consumidor principal
ShipmentCreated	$\emptyset \rightarrow \text{PENDING}$	G9 (notifica al cliente), G10
ShipmentInTransit	$\text{PENDING} \rightarrow \text{IN_TRANSIT}$	G9 (“Tu paquete va en camino”)
ShipmentDelivered	$\text{IN_TRANSIT} \rightarrow \text{DELIVERED}$	G9 (“Tu paquete llegó”), G10
ShipmentCancelled	$\text{PENDING} \rightarrow \text{CANCELLED}$	G9, G5 (confirmar cancelación)
ShipmentFailed	$\text{IN_TRANSIT} \rightarrow \text{FAILED}$	G9 (“Hubo un problema”), G5
ShipmentReturned	$\text{DELIVERED} \rightarrow \text{RETURNED}$	G9, G5, G10

Cuadro 3: Catálogo completo de eventos del servicio de despacho.

6. Conceptos Técnicos Obligatorios

Concepto	Cómo lo demostramos
REST	Exponemos los endpoints POST , GET y PATCH descritos en la Sección 4, siguiendo convenciones REST (verbos HTTP, códigos de estado, recursos en plural).
Eventos	Emitimos eventos (Sección 5) en cada transición de estado, usando webhooks HTTP o el mecanismo acordado con G9.
Consistencia Eventual	El despacho no es instantáneo. El usuario ve PENDING inmediatamente, y los estados cambian de forma asíncrona (simulación temporizada). Los otros grupos se enteran vía eventos, no por consulta síncrona.
Manejo de Errores	Validamos inputs, respondemos con códigos HTTP correctos (400, 404, 409, 422), y contemplamos estados de error (CANCELLED , FAILED , RETURNED).

7. Hoja de Ruta por Fases

⚠ Corrección crítica: Fecha de Fase 3

El briefing original indicaba que la Fase 3 vence el **30 de Julio**, pero la Fase 4 vence el **7 de Julio**. Esto es cronológicamente imposible. La fecha correcta de la Fase 3 es **30 de Junio**.

Fase	Nombre	Vencimiento	Entregables
1	Contratos	16 Jun	Contrato REST (Swagger/Markdown), contrato de eventos (JSON), diagrama de arquitectura, matriz de dependencias.
2	Mock Funcional	23 Jun	Repositorio en GitHub. Servicio desplegado en Render que responde JSON simulado a los endpoints, sin lógica real. Permite al G5 probar su código.
3	Despliegue Real	30 Jun	Servicio funcional (FastAPI o Node.js) en Render, conectado a base de datos en Supabase/Firebase. Tabla shipments con persistencia real.
4	Integración	7 Jul	Conexión real con G5 y G9. Script de simulación temporal: $PENDING \xrightarrow{30s} IN_TRANSIT \xrightarrow{60s} DELIVERED$, emitiendo eventos en cada paso.
5	Defensa Final	13 Jul	Presentación con diapositivas, demo en vivo del flujo completo (pedido → despacho → entrega), defensa oral individual (2 preguntas técnicas por persona).

Cuadro 4: Cronograma corregido del proyecto.

8. Stack Tecnológico Propuesto

Capa	Tecnología	Justificación
Framework	Python + FastAPI	Rápido de desarrollar, tipado, documentación automática (Swagger).
Hosting	Render (Free)	Tier gratuito suficiente para el proyecto.
Base de datos	Supabase (PostgreSQL)	Tier gratuito, interfaz web para inspección, compatible con SQL estándar.
Eventos	Webhooks HTTP	Sin infraestructura adicional; un simple <code>POST</code> a la URL de G9.
Versionado	GitHub	Repositorio del equipo con ramas por fase.

9. Esquema de Base de Datos

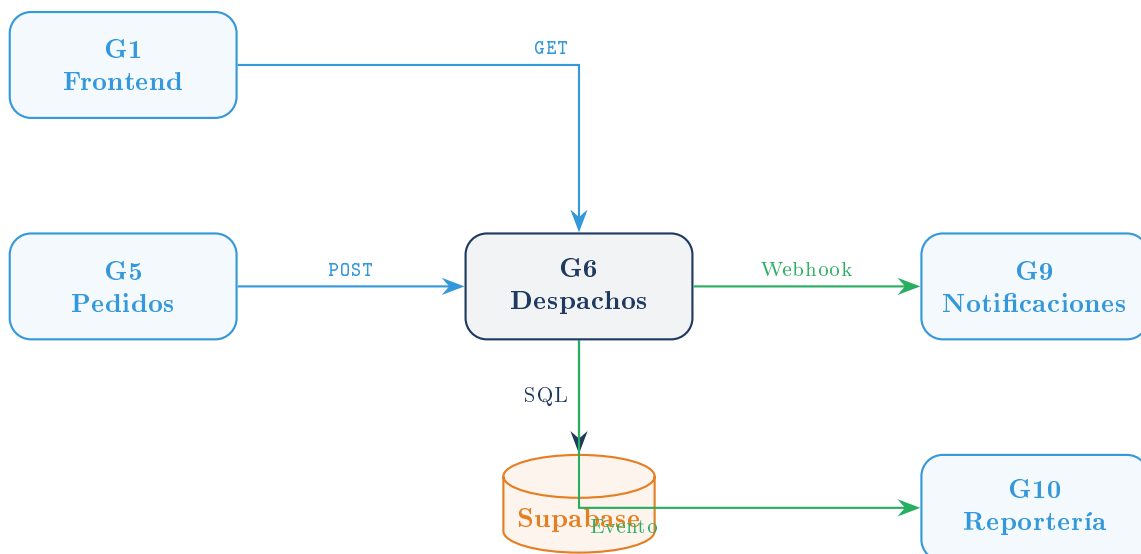
Tabla: shipments

```
{
  "shipment_id": "TEXT PRIMARY KEY (ej: SHP-uuid)",
  "order_id": "TEXT UNIQUE NOT NULL",
  "customer_name": "TEXT NOT NULL",
  "address": "TEXT NOT NULL",
  "city": "TEXT NOT NULL",
  "weight_kg": "REAL NOT NULL CHECK (weight_kg > 0)",
  "status": "TEXT NOT NULL DEFAULT 'PENDING'",
  "created_at": "TIMESTAMP DEFAULT NOW()",
  "updated_at": "TIMESTAMP DEFAULT NOW()",
  "estimated_delivery": "TIMESTAMP"
}
```

i Nota

Usar un CHECK constraint en **status** para que solo acepte los valores válidos: PENDING, IN_TRANSIT, DELIVERED, CANCELLED, FAILED, RETURNED.

10. Diagrama de Arquitectura



11. Resumen de Correcciones Aplicadas

A continuación se listan todas las correcciones y adiciones respecto al briefing original:

- C1. Fecha de Fase 3:** Cambiada de “30 de Julio” a “**30 de Junio**”. La fecha original hacía que la Fase 4 (7 de Julio) venciera antes que la Fase 3.
- C2. Grupo 1 como consumidor:** Agregado G1 (Frontend) a la matriz de interacciones, ya que consume el endpoint GET /shipments/{id} para tracking.
- C3. Estados de error:** Agregados CANCELLED, FAILED y RETURNED al ciclo de vida del envío. El briefing original solo contemplaba el “camino feliz”.

- C4. Endpoint PATCH:** Agregado `PATCH /shipments/{id}` para actualizar el estado del envío. Esencial para la simulación automática de la Fase 4.
- C5. Mecanismo de eventos:** Especificado que usaremos **webhooks HTTP** como mecanismo por defecto. El briefing original no aclaraba la tecnología.
- C6. Payloads de eventos:** Agregados ejemplos concretos del formato JSON de los eventos, con campos `event_type`, `event_id`, `timestamp` y `payload`.
- C7. Esquema de base de datos:** Agregada la definición de la tabla `shipments` con tipos, constraints y valores por defecto.
- C8. Códigos de error detallados:** Documentados los errores HTTP esperados para cada endpoint (400, 404, 409, 422).

12. Próximos Pasos Inmediatos

1. **Hoy (11 Jun):** Cada miembro del equipo lee este documento completo.
2. **Hoy/Mañana:** Contactar a G5 y G9 para acordar contratos JSON.
3. **Antes del 14 Jun:** Generar el documento Swagger/Markdown definitivo con los contratos REST y de eventos.
4. **Antes del 16 Jun:** Entregar Fase 1 (contratos + diagrama de arquitectura + matriz de dependencias).
5. **17–23 Jun:** Crear repo GitHub, implementar el mock en Render y entregar Fase 2.

Consejo

Si tienen dudas sobre algún punto de este documento, discútanlo en el grupo **antes** de asumir cosas. Las decisiones de diseño que tomemos ahora nos ahorrarán reescrituras después.